

CENTRO UNIVERSITARIO CEUNI - FAMETRO

Curso: Tecnólogo em Análise e Desenvolvimento de Sistemas

2026.1

DISCIPLINA: SISTEMAS OPERACIONAIS

Interrupções, Exceções, E/S, Buffering, Spooling, Reentrância
Funções do Núcleo, Modo de Acesso e System Calls

Professor: Alexander Farias

Manaus - AM

OBJETIVOS DE APRENDIZAGEM

Ao final desta aula, o estudante será capaz de:

- Compreender o funcionamento das interrupções e exceções em sistemas operacionais, identificando suas diferenças e aplicações práticas;
- Explicar os mecanismos de operações de Entrada/Saída (E/S), incluindo buffering, spooling e reentrância;
- Identificar as funções do núcleo do sistema operacional e compreender a importância dos modos de acesso;
- Analisar o funcionamento das system calls e sua relação com as rotinas do sistema operacional;
- Aplicar os conceitos teóricos em situações práticas do cotidiano computacional.

OBJETO DE CONHECIMENTO 347

interrupções, Exceções, Operações de Entrada/Saída, Buffering, Spooling e Reentrância

1. Interrupções

As interrupções representam um dos conceitos fundamentais na arquitetura de computadores modernos. De acordo com Tanenbaum (2016), uma interrupção é um sinal eletrônico enviado ao processador por um dispositivo de hardware ou software, indicando que um evento requer atenção imediata.

Para compreender melhor, imagine um professor dando aula para uma turma. De repente, toca o telefone da secretária. O professor precisa interromper sua explicação momentaneamente para atender a chamada, e depois retornar exatamente de onde parou. As interrupções funcionam de forma análoga no computador: o processador está executando tarefas normais quando recebe um sinal de que algo importante aconteceu e precisa ser tratado.

Existem dois tipos principais de interrupções:

- Interrupções de Hardware: Geradas por dispositivos físicos, como teclado, mouse, disco rígido ou placa de rede. Por exemplo, quando você pressiona uma tecla, o controlador do teclado envia uma interrupção ao processador para que o caractere digitado seja processado;
- Interrupções de Software: Também chamadas de traps, são geradas intencionalmente por programas em execução. Um exemplo comum é quando um programa precisa realizar uma operação de E/S e faz uma chamada ao sistema operacional.

EXEMPLO PRÁTICO

Quando você move o mouse, o controlador USB gera uma interrupção aproximadamente 125 vezes por segundo. O processador suspende momentaneamente o que está fazendo, atualiza a posição do cursor na tela e retorna à tarefa anterior. Sem o mecanismo de interrupção, o computador precisaria ficar constantemente verificando se o mouse se moveu, desperdiçando recursos preciosos de processamento.

2. Exceções

Exceções são eventos síncronos que ocorrem durante a execução de uma instrução, geralmente indicando uma condição anormal ou erro. Diferentemente das interrupções, que são assíncronas e podem ocorrer a qualquer momento, as exceções são previsíveis e estão diretamente relacionadas ao fluxo de execução do programa.

Pense em uma exceção como um alarme de incêndio em um prédio. O alarme soa quando há realmente um problema (fogo), não aleatoriamente. Da mesma forma, uma exceção é sinalizada quando algo inesperado acontece durante a execução de um programa.

Os principais tipos de exceções incluem:

- Falta de Página (Page Fault): Ocorre quando um programa tenta acessar uma página de memória virtual que não está carregada na memória física;
- Divisão por Zero: Tentativa de dividir um número por zero;

- **Violação de Acesso:** Quando um programa tenta acessar uma área de memória que não lhe pertence;
- **Instrução Ilegal:** Quando o processador encontra uma instrução que não reconhece.

DIFERENÇA IMPORTANTE

Interrupção: Assíncrona, pode ocorrer a qualquer momento, geralmente gerada por hardware externo. **Execução:** Síncrona, ocorre em resposta a execução de uma instrução específica, geralmente indicando erro ou condição especial.

3. Operações de Entrada/Saída (E/S)

As operações de Entrada/Saída (E/S ou I/O - Input/Output) representam a comunicação entre o computador e o mundo externo. Tanenbaum (2016) destaca que o sistema operacional atua como intermediário entre os dispositivos de hardware e os programas de aplicação, fornecendo abstrações que simplificam o uso desses dispositivos.

Imagine um restaurante: você (programa) faz um pedido ao garçom (sistema operacional), que leva o pedido à cozinha (hardware). O garçom também traz sua comida pronta de volta. Você não precisa saber como a cozinha funciona - apenas faz o pedido e recebe o resultado. O sistema operacional funciona exatamente assim, abstraindo a complexidade do hardware.

Existem três técnicas principais para realizar operações de E/S:

- 1.E/S Programada (Programmed I/O):** O processador envia comandos ao dispositivo e fica verificando constantemente (polling) se a operação foi concluída. É simples mas ineficiente, pois consome tempo de processamento;
- 2.E/S por Interrupção (Interrupt-Driven I/O):** O processador envia o comando e continua executando outras tarefas. Quando o dispositivo termina, gera uma interrupção para notificar o processador;
- 3.E/S com DMA (Direct Memory Access):** Um controlador especializado (controlador DMA) gerencia a transferência de dados diretamente entre o dispositivo e a memória, sem envolvimento constante do processador.

EXEMPLO DO COTIDIANO

Ao copiar um arquivo de 1GB do disco rígido para um pen drive: - Sem DMA: O processador ficaria ocupado durante toda a operação, movendo cada byte individualmente. - Com DMA: O processador inicia a operação e pode executar outras tarefas enquanto o controlador DMA gerencia a transferência. Quando termina, uma interrupção notifica o processador.

4. Buffering

Buffering é uma técnica que utiliza uma área temporária de memória (buffer) para armazenar dados durante transferências entre dispositivos com diferentes velocidades. O objetivo principal é reduzir a diferença de velocidade entre componentes do sistema.

Um exemplo cotidiano é o buffer de vídeo do YouTube. Quando você assiste a um vídeo, o player baixa alguns segundos (ou minutos) à frente do que você está vendo. Essa área pré-carregada é o buffer. Se sua conexão ficar lenta momentaneamente, o vídeo continua rodando usando os dados do buffer, evitando travamentos.

Os benefícios do buffering incluem:

- Equalização de Velocidade: Permite que dispositivos rápidos e lentos comuniquem-se eficientemente;
- Desacoplamento: O produtor de dados não precisa esperar pelo consumidor;
- Suavização de Picos: Absorve variações momentâneas na taxa de transferência.

TIPOS DE BUFFER

Buffer Simples: Uma única área de memória. Enquanto um lado preenche, o outro consome. Buffer Duplo: Duas áreas alternadas. Uma é preenchida enquanto a outra é consumida, permitindo fluxo contínuo. Buffer Circular: Várias áreas organizadas em círculo, ideal para fluxos contínuos de dados como áudio e vídeo.

5. Spooling

Spooling (Simultaneous Peripheral Operations On-Line) é uma técnica que permite que um dispositivo de E/S seja compartilhado entre múltiplos processos, criando a ilusão de que cada um tem acesso exclusivo ao dispositivo.

O spooling funciona como uma fila de espera inteligente. Imagine um restaurante popular onde você pega uma senha e espera sentado até ser chamado. O restaurante continua atendendo outros clientes enquanto você espera confortavelmente. O spooling faz o mesmo com dispositivos como impressoras.

O funcionamento do spooling envolve:

- Um processo gera saída para um arquivo em disco (spool) em vez de enviar diretamente ao dispositivo;
- O arquivo é colocado em uma fila de espera;
- Um processo especial (daemon de spooling) gerencia a fila e envia os dados ao dispositivo na ordem correta;
- O processo original pode continuar executando imediatamente, sem esperar.

EXEMPLO PRÁTICO

Em uma empresa com uma única impressora laser: - Usuário A envia um documento de 100 páginas para imprimir - Usuário B envia um documento de 2 páginas - Sem spooling: Usuário B precisa esperar todas as 100 páginas do Usuário A - Com spooling: Ambos enviam para a fila. O documento do Usuário B pode ser impresso primeiro se o administrador configurar prioridades, ou logo após terminar o atual, sem que os usuários precisem esperar ociosamente.

6. Reentrância

Reentrância é a propriedade de um programa ou rotina que permite ser interrompida durante sua execução e depois ser chamada novamente (reentrada) antes que a execução anterior seja concluída, sem que haja conflito entre as diferentes execuções.

Pense em uma função reentrante como uma receita de bolo que pode ser usada simultaneamente por várias pessoas em cozinhas diferentes. Cada pessoa segue a mesma receita, mas usa seus próprios ingredientes e utensílios. Uma pessoa não interfere no bolo da outra, mesmo usando a mesma receita.

Uma rotina é reentrante quando:

- Não modifica seu próprio código;
- Não mantém variáveis globais ou estáticas que possam ser alteradas por diferentes chamadas;
- Trabalha apenas com parâmetros passados e variáveis locais na pilha;
- Cada chamada tem seu próprio conjunto de variáveis.

IMPORTANCIA DA REENTRANCIA

Em sistemas operacionais multitarefa, vários processos podem precisar usar a mesma rotina do sistema simultaneamente. A reentrância garante que: - O código do sistema operacional possa ser compartilhado entre processos - Não haja corrupção de dados quando interrupções ocorrem - O sistema seja mais eficiente, compartilhando código comum. Exemplo: A rotina de leitura de arquivo do sistema operacional pode ser usada por vários programas ao mesmo tempo sem conflitos.

ATIVIDADES DISCURSIVAS - OBJETO DE CONHECIMENTO 347

ATIVIDADE

1

Explique, usando exemplos do cotidiano, a diferença entre interrupções e exceções. De um exemplo prático de situação em que cada uma ocorreria em um sistema computacional.

ATIVIDADE

2

Descreva como o mecanismo de DMA melhora o desempenho do sistema. Compare uma operação de cópia de arquivo com e sem DMA, explicando as vantagens e desvantagens de cada abordagem.

ATIVIDADE

3

Um usuário está assistindo a um vídeo em streaming e percebe que o vídeo

para frequentemente para carregar (buffering). Explique o que esta acontecendo tecnicamente e proponha solucoes para minimizar esse problema, considerando os conceitos de buffering estudados.

ATIVIDADE

4

Em um laboratorio de informatica com 20 computadores e apenas 2 impressoras, como o spooling ajuda a organizar as impressoes? Explique o fluxo completo desde o momento em que um aluno clica em 'Imprimir' ate o documento sair na impressora.

ATIVIDADE

5

Por que a reentrancia e fundamental para rotinas de sistema operacional? De um exemplo de problema que poderia ocorrer se uma rotina do sistema operacional nao fosse reentrante e duas aplicacoes a chamassem simultaneamente.

OBJETO DE CONHECIMENTO 348

Funcoes do Nucleo, Modo de Acesso, Rotinas do Sistema Operacional e System Calls

1. Funcoes do Nucleo (Kernel)

O nucleo (kernel) e o coracao do sistema operacional - a parte central que opera em modo privilegiado e tem acesso direto a todo o hardware. Tanenbaum (2016) descreve o nucleo como a camada de software que gerencia os recursos do computador e fornece servicos aos programas de aplicacao.

Imagine um shopping center: o nucleo seria como a administracao do shopping. Ele nao vende produtos diretamente (isso e feito pelas lojas/aplicacoes), mas gerencia tudo: seguranca, limpeza, energia, agua, estacionamento - todos os recursos compartilhados que as lojas precisam para funcionar.

As principais funcoes do nucleo incluem:

- Gerenciamento de Processos: Criacao, escalonamento, sincronizacao e terminacao de processos;
- Gerenciamento de Memoria: Alocacao e desalocacao de memoria para processos, implementacao de memoria virtual;
- Gerenciamento de Arquivos: Criacao, leitura, escrita e organizacao de arquivos em dispositivos de armazenamento;
- Gerenciamento de Dispositivos: Controle de dispositivos de E/S atraves de drivers;
- Seguranca e Protecao: Controle de acesso a recursos, autenticacao de usuarios;
- Comunicacao entre Processos: Mecanismos para que processos troquem informacoes.

ARQUITETURAS DO NUCLEO

Monolitico: Todo o codigo do nucleo executa em modo privilegiado como um unico programa grande (ex: Linux tradicional). Micronucleo: Apenas funcoes essenciais no

nucleo; outros servicos executam como processos em modo usuario (ex: MINIX, QNX). Hibrido: Combina elementos de ambos (ex: Windows NT, Linux moderno com modulos).

2. Modo de Acesso

Os processadores modernos operam em pelo menos dois modos distintos: modo nucleo (kernel mode) e modo usuario (user mode). Essa dualidade e fundamental para a seguranca e estabilidade do sistema.

Pense nos modos de acesso como niveis de seguranca em um aeroporto. A area publica (modo usuario) e onde os passageiros circulam - tem acesso limitado. A area restrita (modo nucleo) e onde so pessoas autorizadas podem entrar, com acesso a todas as areas.

Caracteristicas do Modo Nucleo:

- Acesso completo a todas as instrucoes do processador;
- Acesso irrestrito a toda a memoria;
- Capacidade de executar instrucoes privilegiadas;
- Controle direto de dispositivos de hardware.

Caracteristicas do Modo Usuario:

- Acesso limitado a um subconjunto de instrucoes;
- Acesso apenas a memoria alocada ao processo;
- Proibicao de executar instrucoes privilegiadas;
- Acesso ao hardware apenas atraves do sistema operacional.

PROTECAO POR HARDWARE

A separacao entre modos e implementada por hardware no processador. Quando uma aplicacao em modo usuario tenta executar uma instrucao privilegiada, o hardware gera uma excecao (trap), transferindo o controle para o sistema operacional, que pode entao tomar acoes corretivas (como terminar o programa mal comportado). Isso

impede que um programa com erro ou malicioso: - Acesse memória de outros programas - Corrompa o sistema operacional - Danifique o hardware

3. Rotinas do Sistema Operacional

As rotinas do sistema operacional são funções ou procedimentos que implementam os serviços oferecidos pelo sistema. Elas podem ser categorizadas em diferentes tipos, dependendo de sua função e do nível em que operam.

Continuando a analogia do shopping, as rotinas seriam como os procedimentos internos da administração: como registrar uma nova loja, como processar uma reclamação, como autorizar acesso ao estacionamento, etc.

Principais categorias de rotinas:

- Rotinas de Gerenciamento de Processos: Criação, terminação, escalonamento e sincronização de processos;
- Rotinas de Gerenciamento de Memória: Alocação, desalocação, paginação e segmentação;
- Rotinas de Sistema de Arquivos: Abertura, leitura, escrita, fechamento e organização de arquivos;
- Rotinas de E/S: Comunicação com dispositivos através de drivers;
- Rotinas de Rede: Comunicação entre computadores;
- Rotinas de Segurança: Autenticação, autorização e auditoria.

BIBLIOTECAS DE SISTEMA

Muitas funcionalidades do sistema operacional são acessadas através de bibliotecas de sistema (system libraries), como a glibc no Linux ou a C Runtime no Windows. Essas bibliotecas fornecem funções de alto nível que encapsulam as chamadas ao sistema, tornando o desenvolvimento mais fácil e portátil. Exemplo: A função `printf()` da linguagem C é uma rotina de biblioteca que, internamente, faz chamadas ao sistema para escrever na tela.

4. System Calls (Chamadas de Sistema)

System calls são a interface programável entre os processos em modo usuário e os serviços do sistema operacional em modo núcleo. Elas representam o mecanismo através do qual os programas solicitam serviços ao sistema operacional.

Imagine que você está em um restaurante e precisa de mais água. Você não vai direto à cozinha pegar (não tem permissão), mas chama o garçom (system call) que leva seu pedido à cozinha (núcleo) e traz o que você precisa.

O processo de uma system call envolve:

1. O programa em modo usuário coloca parâmetros em registradores ou na pilha;
2. O programa executa uma instrução especial (TRAP ou SYSCALL) que causa uma interrupção;
3. O processador muda para modo núcleo e transfere controle para uma rotina do sistema operacional;
4. O sistema operacional verifica os parâmetros e executa o serviço solicitado;
5. O controle retorna ao programa em modo usuário, com o resultado da operação.

EXEMPLOS DE SYSTEM CALLS

No Linux: - `fork()`: Cria um novo processo - `execve()`: Executa um programa - `open()`: Abre um arquivo - `read()`: Lê dados de um arquivo - `write()`: Escreve dados em um arquivo - `close()`: Fecha um arquivo - `exit()`: Termina um processo No Windows (API Win32): - `CreateProcess()`: Cria um novo processo - `CreateFile()`: Cria ou abre um arquivo - `ReadFile()`: Lê dados de um arquivo - `WriteFile()`: Escreve dados em um arquivo - `ExitProcess()`: Termina um processo

CUSTO DAS SYSTEM CALLS

Cada system call envolve uma mudança de contexto entre modo usuário e modo núcleo, o que tem um custo computacional significativo. Por isso: - System calls são mais lentas que chamadas de função normais - Sistemas bem projetados minimizam o número de system calls necessárias - Técnicas como batching (agrupar várias operações) ajudam a reduzir overhead Exemplo: Em vez de fazer 1000 `writes()` de 1 byte, é muito mais eficiente fazer 1 `write()` de 1000 bytes.

ATIVIDADES DISCURSIVAS - OBJETO DE CONHECIMENTO 348

ATIVIDADE 1

Explique por que a separação entre modo núcleo e modo usuário é essencial para a segurança e estabilidade de um sistema operacional. De um exemplo prático de problema que poderia ocorrer se todos os programas executassem em modo núcleo.

ATIVIDADE 2

Descreva o fluxo completo de uma system call para abrir um arquivo, desde o momento em que um programa chama `fopen()` na linguagem C até o arquivo ser realmente aberto pelo sistema operacional. Inclua as mudanças de modo de execução.

ATIVIDADE 3

Compare as arquiteturas de núcleo monolítico e micronúcleo, apresentando vantagens e desvantagens de cada uma. Em que tipo de sistema cada arquitetura seria mais adequada? Justifique sua resposta.

ATIVIDADE 4

Um desenvolvedor está criando um programa que precisa ler 1 milhão de registros de um arquivo. Ele pode ler registro por registro (1 milhão de reads) ou ler blocos maiores (100 reads de 10 mil registros). Explique qual abordagem é mais eficiente e por que, considerando o conceito de system calls.

ATIVIDADE 5

Em um sistema operacional moderno, por que as bibliotecas de sistema são importantes? Explique a relação entre uma função de biblioteca como `printf()` e a system call `write()`, descrevendo como elas trabalham juntas.

REFERENCIAS BIBLIOGRAFICAS

TANENBAUM, A. S.; BOS, H. Sistemas Operacionais Modernos. 4. ed. Sao Paulo: Pearson Education do Brasil, 2016.

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. Sistemas Operacionais: Conceitos e Aplicacoes. Rio de Janeiro: Campus, 2000.

STALLINGS, W. Sistemas Operacionais: Projeto e Implementacao. Rio de Janeiro: Prentice-Hall, 2001.

DEITEL, H. M.; DEITEL, P. J.; CHOFFNES, D. R. Sistemas Operacionais. 3. ed. Sao Paulo: Pearson Prentice Hall, 2005.